TASK 1

AIM:To find the contiguous subarray with the largest sum using Kadane's Algorithm.

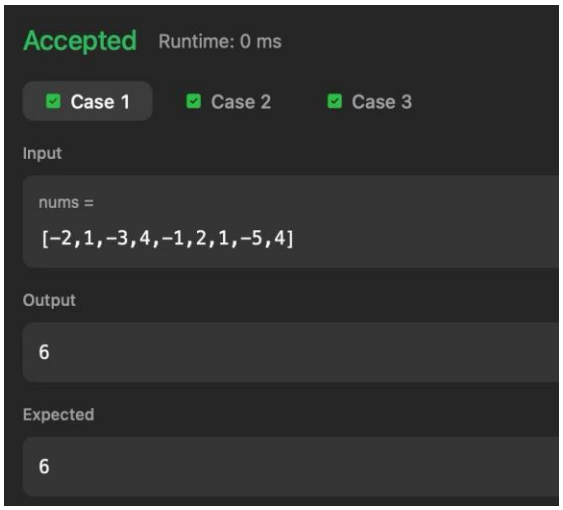
ALGORITHM:

1. Initialize maxSum and currentSum with first element.
2. Traverse array from index 1.
3. Update currentSum = max(arr[i], currentSum + arr[i]).
4. Update maxSum = max(maxSum, currentSum).
5. Return maxSum.

PROGRAM

```
class Solution {
   public int maxSubArray(int[] nums) {
      int maxSum = nums[0];
      int currentSum = nums[0];

      for (int i = 1; i < nums.length; i++) {
         currentSum = Math.max(nums[i], currentSum + nums[i]);
         maxSum = Math.max(maxSum, currentSum);
      }
      return maxSum;
   }
}
```

OUTPUT



RESULT:Thus the program for Maximum Subarray was executed successfully.

TASK 2

AIM:To find the number of contiguous segments whose sum equals given value d.

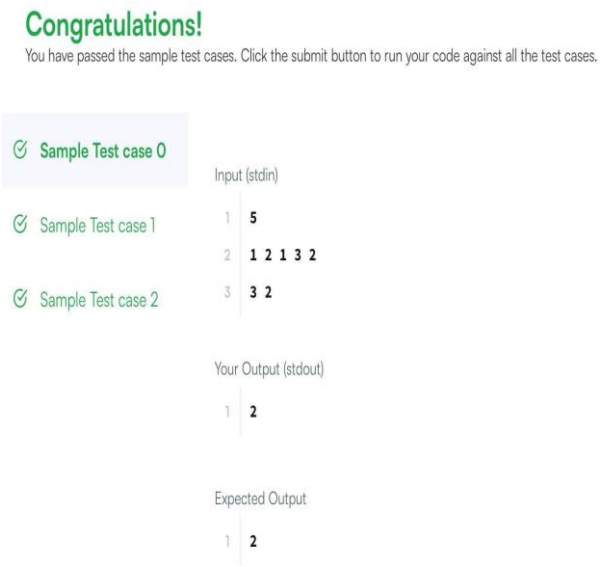
ALGORITHM:

1. Initialize count = 0.
2. Traverse from index 0 to n-m.
3. Find sum of next m elements.
4. If sum == d, increment count.
5. Return count.

PROGRAM

```
import java.io.*;
import java.util.*;

public class Solution {

    public static int birthday(List<Integer> s, int d, int m) {
        int count = 0;

        for (int i = 0; i <= s.size() - m; i++) {
            int sum = 0;
            for (int j = i; j < i + m; j++) {
                sum += s.get(j);
            }

            if (sum == d) {
                count++;
            }
        }

        return count;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        List<Integer> s = new ArrayList<Integer>();

        for (int i = 0; i < n; i++) {
            s.add(sc.nextInt());
        }

        int d = sc.nextInt();
        int m = sc.nextInt();

        int result = birthday(s, d, m);
        System.out.println(result);

        sc.close();
    }
}
```

OUTPUT:



RESULT:Thus the program for Birthday Bar was executed successfully

TASK 3

AIM:To find maximum contiguous and non-contiguous subarray sum.

ALGORITHM

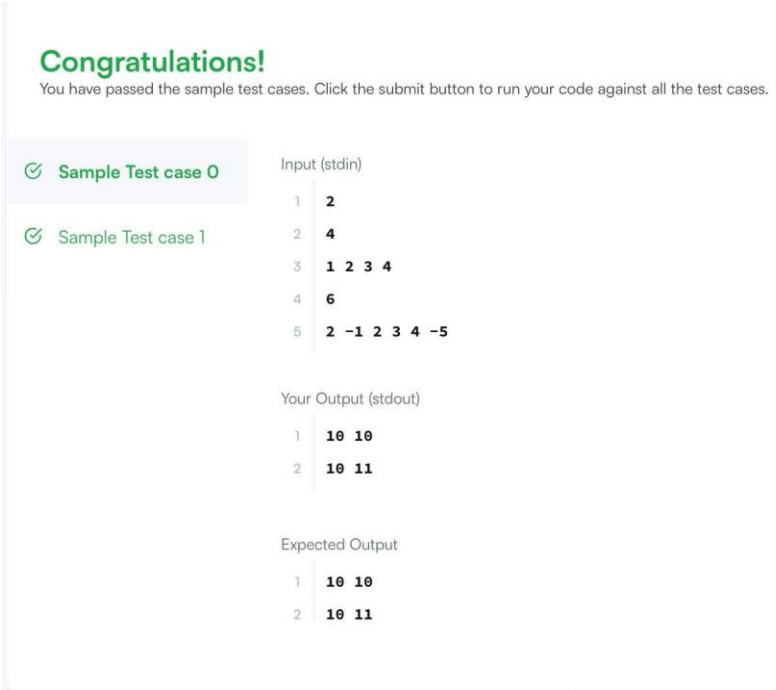
1. Start from first element.
2. Keep adding elements.
3. If sum becomes smaller than current element → start new sum.
4. Keep track of maximum value.

PROGRAM

```
import java.io.*;
import java.util.*;

public class Solution {

    public static int[] maxSubarray(int[] arr) {
        int currentMax = arr[0];
        int globalMax = arr[0];

        for (int i = 1; i < arr.length; i++) {
            currentMax = Math.max(arr[i], currentMax + arr[i]);
            globalMax = Math.max(globalMax, currentMax);
        }
        int subsequenceSum = 0;
        int maxElement = arr[0];

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > 0) {
                subsequenceSum  +=  arr[i];
            }
            maxElement = Math.max(maxElement, arr[i]);
        }
        if (subsequenceSum == 0) {
            subsequenceSum = maxElement;
        }

        return new int[]{globalMax, subsequenceSum};
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();   // number of test cases

        while (t-- > 0) {

            int n = sc.nextInt();
            int[] arr = new int[n];

            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }

            int[] result = maxSubarray(arr);
```

```
            System.out.println(result[0] + " " + result[1]);
        }

        sc.close();
    }
}.
```

OUTPUT



RESULT:Thus the program for Maximum Subarray (Contiguous & Non-Contiguous) was executed successfully.

TASK 4

AIM:To find maximum sum of circular subarray.

ALGORITHM:

1. Find normal max subarray sum using Kadane.
2. Find total array sum.
3. Find minimum subarray sum using Kadane.
4. Circular sum = total - minSum.
5. Return max(normalSum, circularSum).

PROGRAM:

```
class Solution {
   public int maxSubarraySumCircular(int[] nums) {

      int totalSum = 0;
      int currentMax = 0;
      int maxSum = nums[0];
      int currentMin = 0;
      int minSum = nums[0];
      for (int num : nums) {

         currentMax = Math.max(num, currentMax + num);
         maxSum = Math.max(maxSum, currentMax);
         currentMin = Math.min(num, currentMin + num);
         minSum = Math.min(minSum, currentMin);
         totalSum += num;
      }

      if (maxSum < 0) {
         return maxSum;
      }

      return Math.max(maxSum, totalSum - minSum);}}
```

OUTPUT:

RESULT:Thus the program for Maximum Sum Circular Subarray was executed successfully.

TASK 5

AIM:To convert a string into a 32-bit signed integer.

ALGORITHM:

1. Remove leading spaces.
2. Check sign.
3. Convert digits to number.
4. Handle overflow.
5. Return result.

PROGRAM

```
class Solution {
   public int myAtoi(String s) {
      int i = 0, n = s.length();
      while (i < n && s.charAt(i) == ' ') i++;
      int sign = 1;
      if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
         if (s.charAt(i) == '-') sign = -1;
         i++;
      }
      long result = 0;
      while (i < n && Character.isDigit(s.charAt(i))) {
         result = result * 10 + (s.charAt(i) - '0');
         if (sign == 1 && result > Integer.MAX_VALUE)
            return Integer.MAX_VALUE;
         if (sign == -1 && -result < Integer.MIN_VALUE)
            return Integer.MIN_VALUE;
         i++;
      }
      return (int)(sign * result);
   }
}
```

OUTPUT:

RESULT:Thus the program for String to Integer (atoi) was executed successfully.

TASK 6

AIM:To find minimum deletions required so that no two adjacent characters are same.

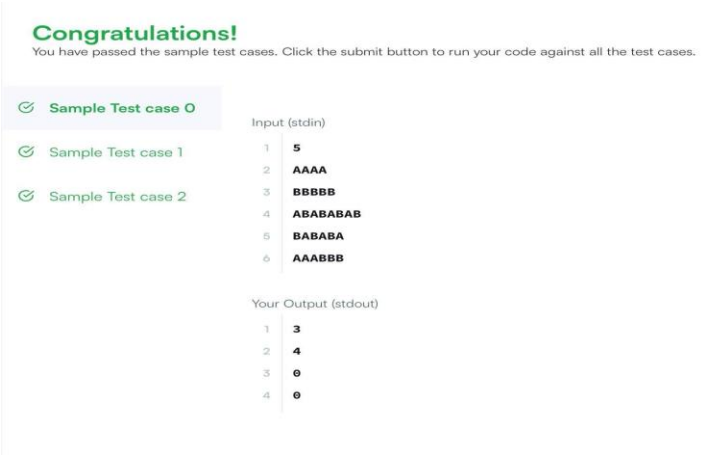
ALGORITHM:

1. Initialize count = 0.
2. Traverse string from index 1.
3. If current character equals previous, increment count.
4. Return count.

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static int alternatingCharacters(String s) {
        int deletions = 0;
       for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                deletions++;
            }
        }
         return deletions;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int q = sc.nextInt();
        sc.nextLine();
        while (q-- > 0) {
            String s = sc.nextLine();
            System.out.println(alternatingCharacters(s));
        }
          sc.close();
    }
}
```

OUTPUT:



RESULT:Thus the program for Alternating Characters was executed successfully.

TASK-07

AIM:To find the length of the longest substring in a given string without repeating characters.

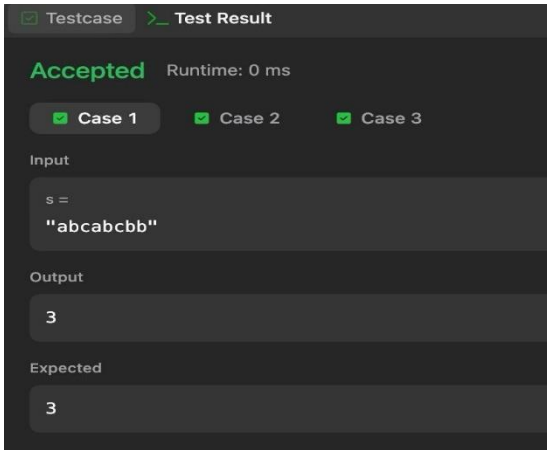
ALGORITHM

1. Initialize a HashSet to store unique characters.
2. Use two pointers: left = 0 and right = 0.
3. Traverse the string using right pointer:
4. If character is not in set → add it.
5. If duplicate found → remove characters from left until duplicate is removed.
6. Update maximum length as:
7. maxLength = max(maxLength, right - left + 1)
8. Return maxLength.

PROGRAM

```
class Solution {
 public int lengthOfLongestSubstring(String s) { Set<Character>set=new HashSet<>();
 int maxLength=0; int left=0;
 for(int right=0;right<s.length();right++){ if(!set.contains(s.charAt(right))){
 set.add(s.charAt(right)); maxLength=Math.max(maxLength,right-left+1);

 }else{
 while(s.charAt(left)!=s.charAt(right)){ set.remove(s.charAt(left));
 left++;
 }
 set.remove(s.charAt(left));left++; set.add(s.charAt(right));
 return maxLength;}
 }
```

OUTPUT

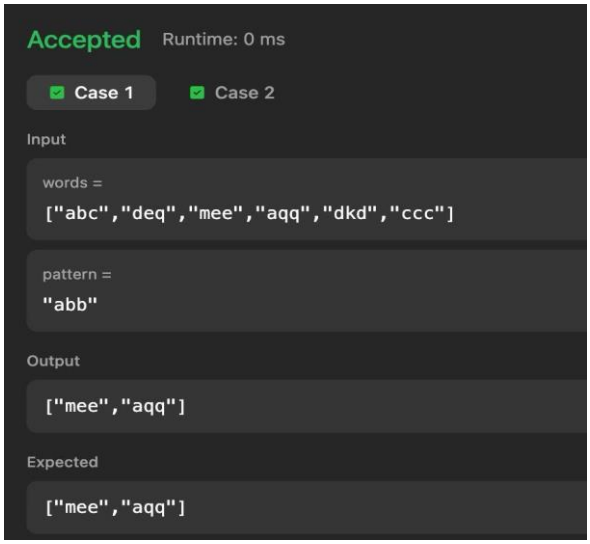


RESULT: Thus is an program for length of the longest substring in a given string without repeating character has been executed successfully ..

1. For each word in the list:
2. Create two maps:
3. Pattern → Word ,Word → Pattern
4. If mapping exists, check consistency.
5. If not, create mapping.
6. If mapping is valid for all characters → add word to result.
7. Return the result list.

PROGRAM

```
class Solution {

 public List<String> findAndReplacePattern(String[] words, String pattern) { List<String> res =
 new ArrayList<>();

 for (String word : words) {

 if (check(word, pattern)) res.add(word);

 }

 return res;

 }


 boolean check(String a, String b) { for (int i = 0; i < a.length(); i++) {

 if (a.indexOf(a.charAt(i)) != b.indexOf(b.charAt(i))) return false;

 }

 return true;

 }

}
```



RESULT: Thus is an program for all words that match a given pattern based on a bijective mapping of characters has been executed successfully

1. Initialize an empty result list.
2. Compare each word with every other word:
3. If word[i] is found inside word[j] (i ≠ j):
4. Add word[i] to result.
5. Use contains() method for substring check.
6. Return the result list.

PROGRAM

```
class Solution {
 public List<String> stringMatching(String[] words) { List<String> result = new ArrayList<>();
 int n = words.length; for(int i = 0; i < n; i++) {
 for(int j = 0; j < n; j++) {
 if(i != j && words[j].contains(words[i])) { result.add(words[i]);
 break;
 }
 }
 }
 return result;
 }
}
```

OUTPUT

RESULT: Thus an program for strings in an array that are substrings of another string in the same array has been executed successfully.